



NGUYÊN LÝ HỆ ĐIỀU HÀNH

Phần 7: Quản lý bộ nhớ

NGUYỄN THỊ HẬU

Khoa Công Nghệ Thông Tin
Đại học Công Nghệ - Đại học quốc gia Hà Nội

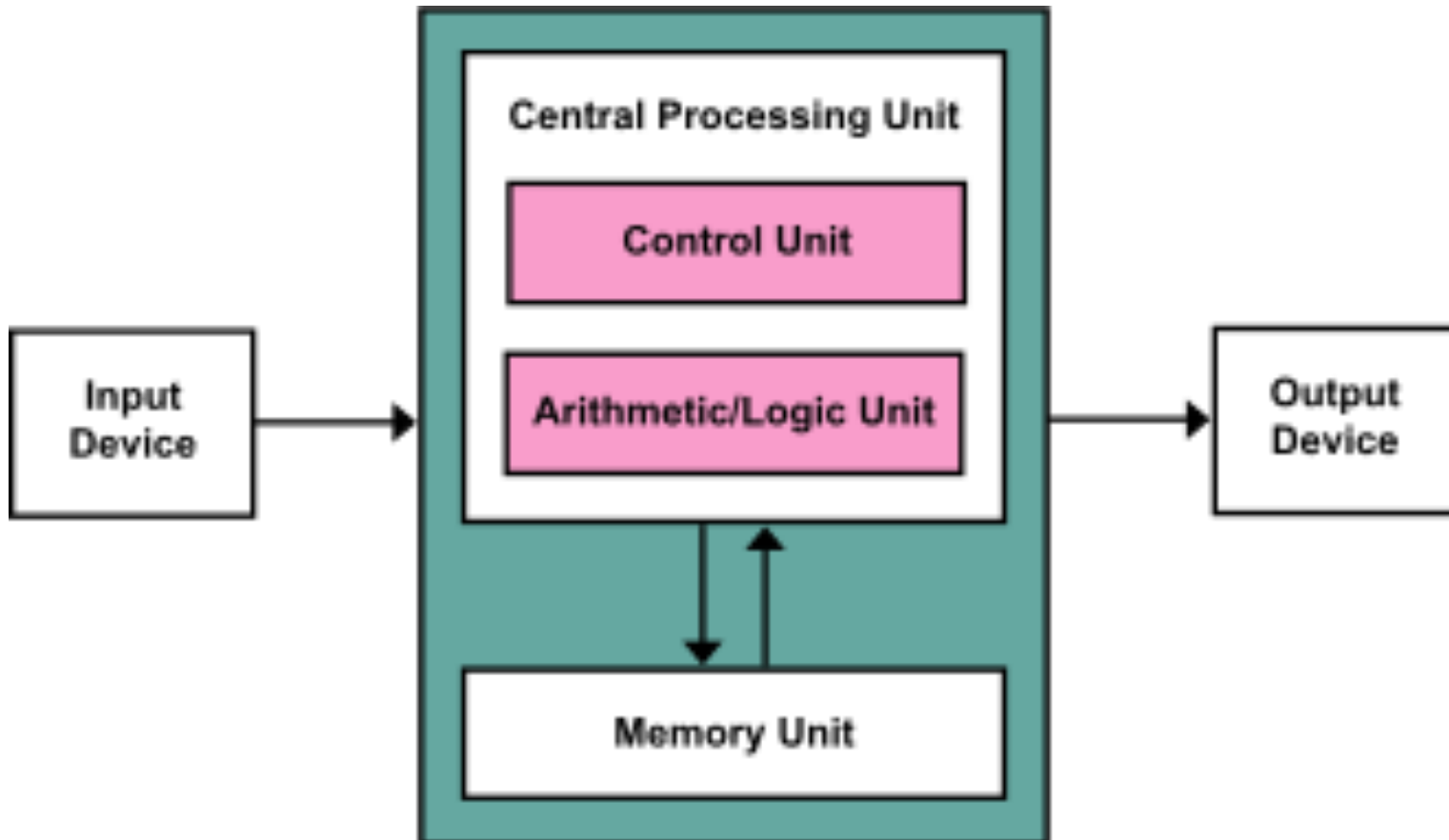
Phần 7: Quản lý bộ nhớ

1. Giới thiệu
2. Swapping
3. Cấp phát liên tục
4. Phân đoạn
5. Phân trang
6. Cấu trúc bảng phân trang

Phần 7: Quản lý bộ nhớ

7.1 Giới thiệu

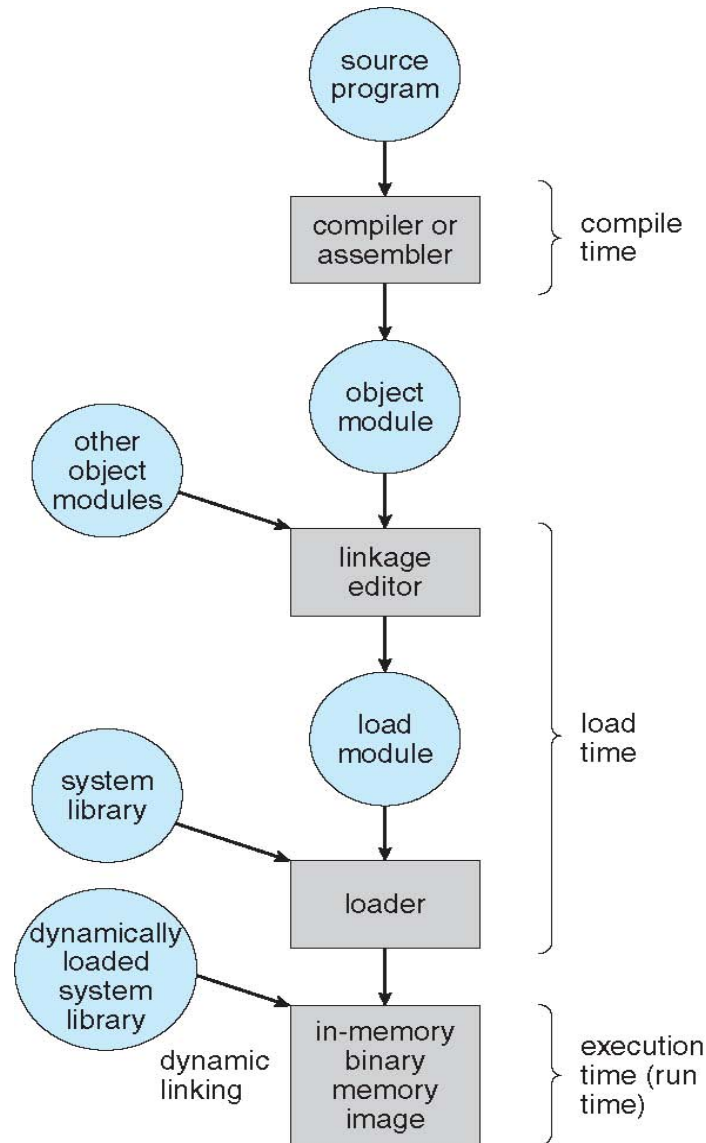
Giới thiệu



Giới thiệu

- Chương trình được HĐH tải từ đĩa vào bộ nhớ, tạo tiến trình để thực thi
- **Input queue**: hàng chờ các tiến trình trên đĩa đang chờ được đưa vào bộ nhớ để thực thi
- CPU chỉ có thể truy cập trực tiếp vào **bộ nhớ chính** và **các thanh ghi**
- CPU truy cập vào thanh ghi tối đa trong một đơn vị thời gian của đồng hồ CPU
- Thời gian CPU truy cập vào bộ nhớ chính có thể trong vài đơn vị thời gian của đồng hồ CPU
- **Cache**, thường đặt trong CPU chip, và là trung gian giữa bộ nhớ chính và các thanh ghi CPU

Các bước xử lý khi chạy một chương trình

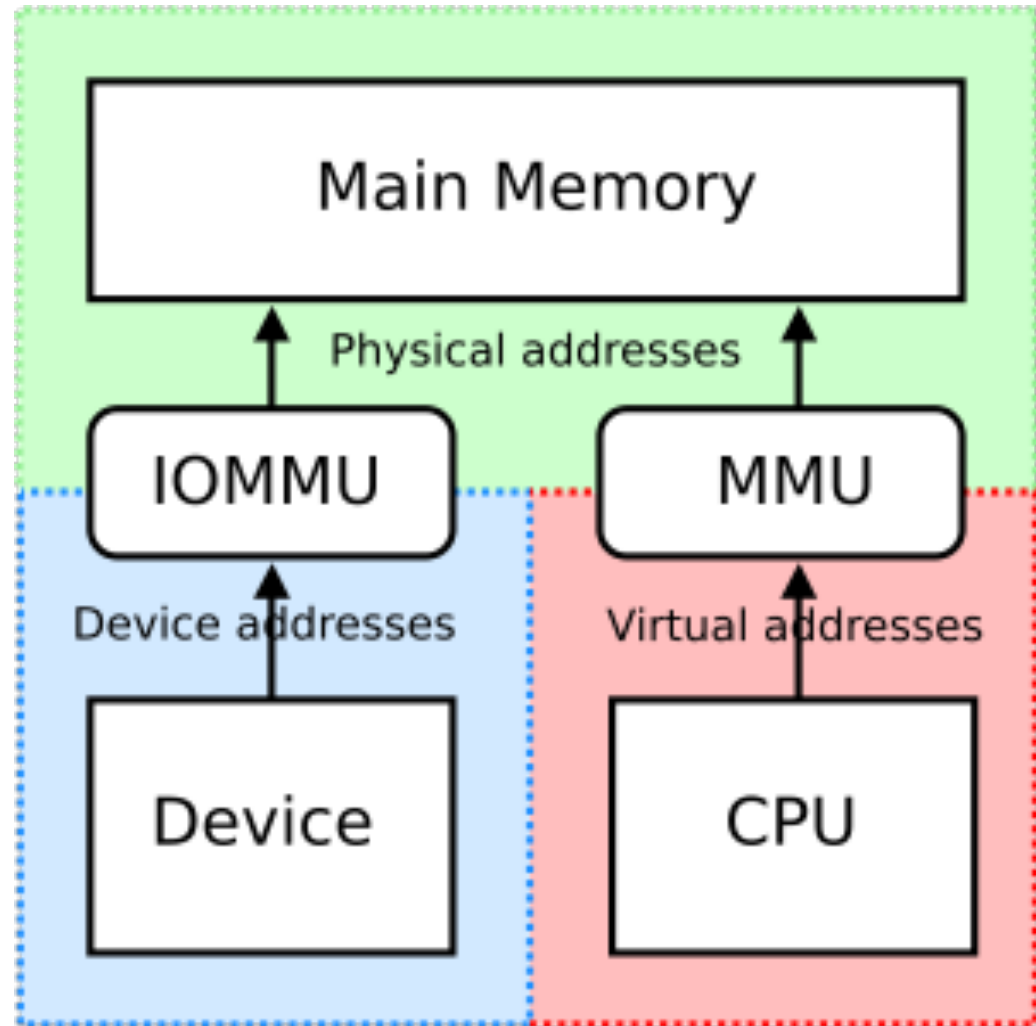


Liên kết địa chỉ

- Trước khi chạy, chương trình trải qua nhiều giai đoạn, địa chỉ của chương trình có thể ở các dạng khác nhau trong mỗi giai đoạn
- Các câu lệnh và dữ liệu cơ thể được liên kết với địa chỉ bộ nhớ ở bất cứ giai đoạn nào:
 - **Giai đoạn biên dịch:** Nếu biết trước địa chỉ bộ nhớ của tiến trình thì có thể tạo ra **mã tuyệt đối**, khi địa chỉ bộ nhớ thay đổi thì cần biên dịch lại chương trình (vd chương trình định dạng .COM của MS-DOS)
 - **Giai đoạn tải:** Nếu địa bộ nhớ chưa biết trong giai đoạn biên dịch thì chương trình biên dịch cần tạo ra **mã cho phép định vị lại**
 - **Giai đoạn thực thi:** Liên kết địa chỉ có thể trì hoãn đến lúc chạy nếu tiến trình có thể bị di chuyển từ phân đoạn này sang phân đoạn khác của bộ nhớ trong thời gian thực thi

Địa chỉ vật lý vs địa chỉ logic

- **Địa chỉ logic** là địa chỉ do CPU tạo ra (*địa chỉ ảo*)
- **Địa chỉ vật lý** là địa chỉ được tải vào các thanh ghi địa chỉ-bộ nhớ của bộ nhớ



Địa chỉ vật lý vs địa chỉ logic

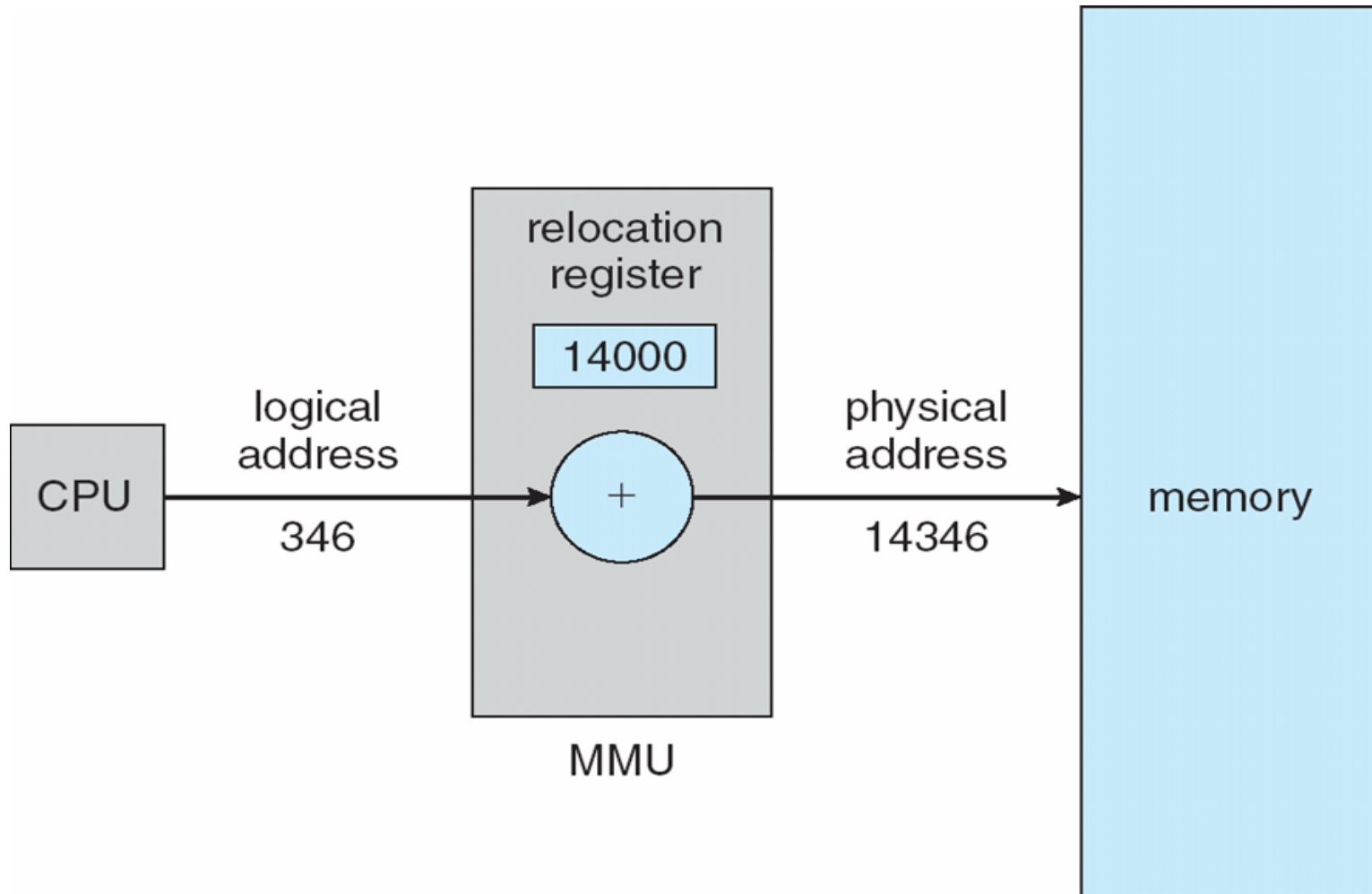
- Trong giai đoạn biên dịch và giai đoạn tải, phương thức liên kết địa chỉ tạo ra địa chỉ vật lý và logic giống nhau
- Giai đoạn thực thi, địa chỉ vật lý và logic có thể khác nhau
- **Không gian địa chỉ logic**: tập tất cả các địa chỉ logic tạo bởi chương trình
- **Không gian địa chỉ vật lý**: tập tất cả các địa chỉ vật lý tạo bởi chương trình

Đơn vị quản lý bộ nhớ (MMU)

- Là bộ phận phần cứng ánh xạ địa chỉ ảo với địa chỉ vật lý trong thời gian chạy
- Có nhiều cách thức ánh xạ
 - MS-DOS trên Intel 80x86 sử dụng 4 thanh ghi định vị lại
- Chương trình của người dùng chỉ làm việc với địa chỉ logic

Phương thức sử dụng thanh ghi định vị lại

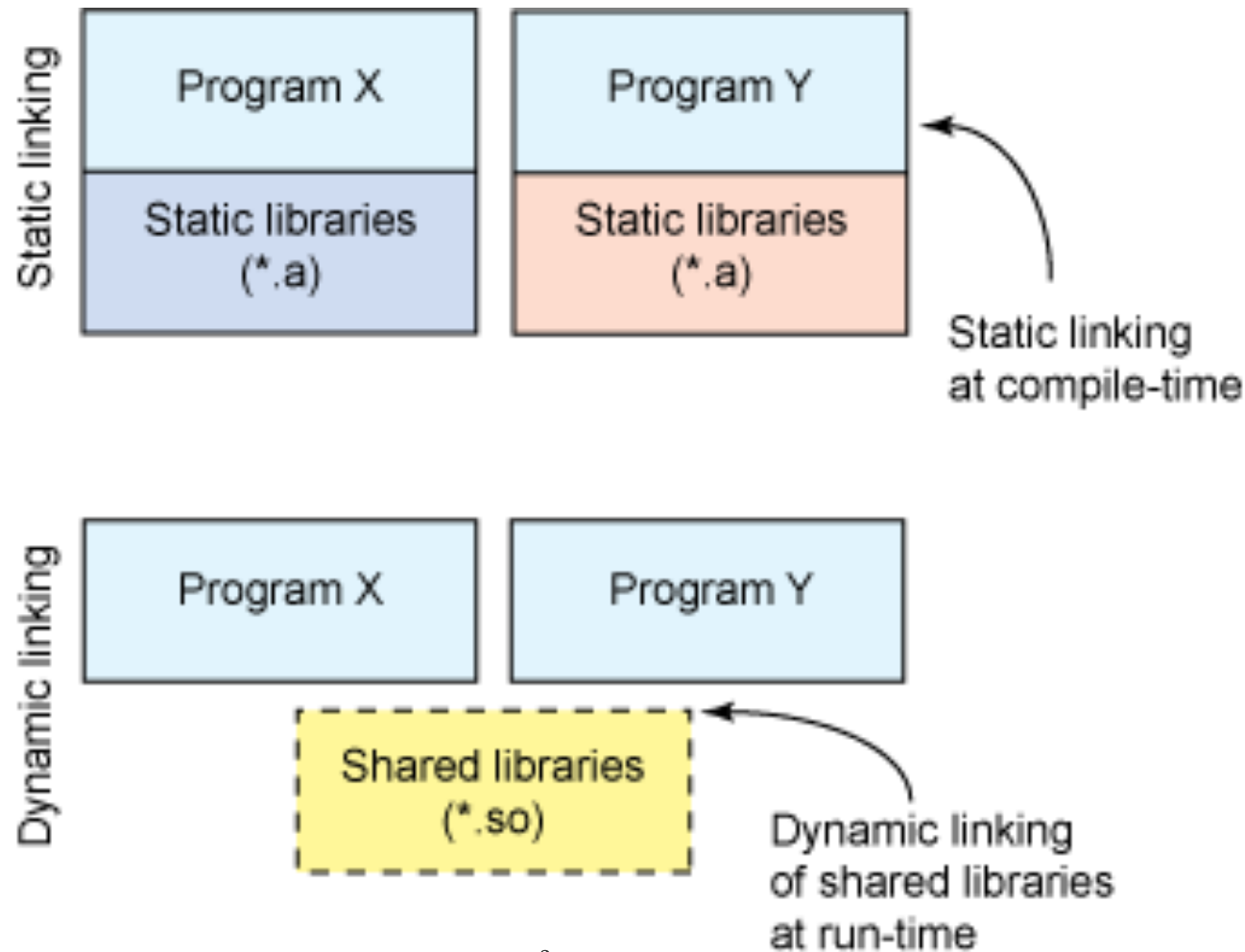
11



Nạp chương trình động

- Hàm chỉ được nạp khi nó được gọi
 - Các hàm được lưu trên đĩa ở định dạng cho phép định vị lại
 - Chương trình chính được tải vào trong bộ nhớ và thực thi
- Ưu điểm:
 - Tối ưu việc sử dụng bộ nhớ
 - Có ích trong trường hợp có đoạn mã dài để xử lý một số trường hợp hiếm khi xảy ra (bất lỗi)
 - Không cần hỗ trợ đặc biệt từ HĐH

Liên kết tĩnh vs liên kết động

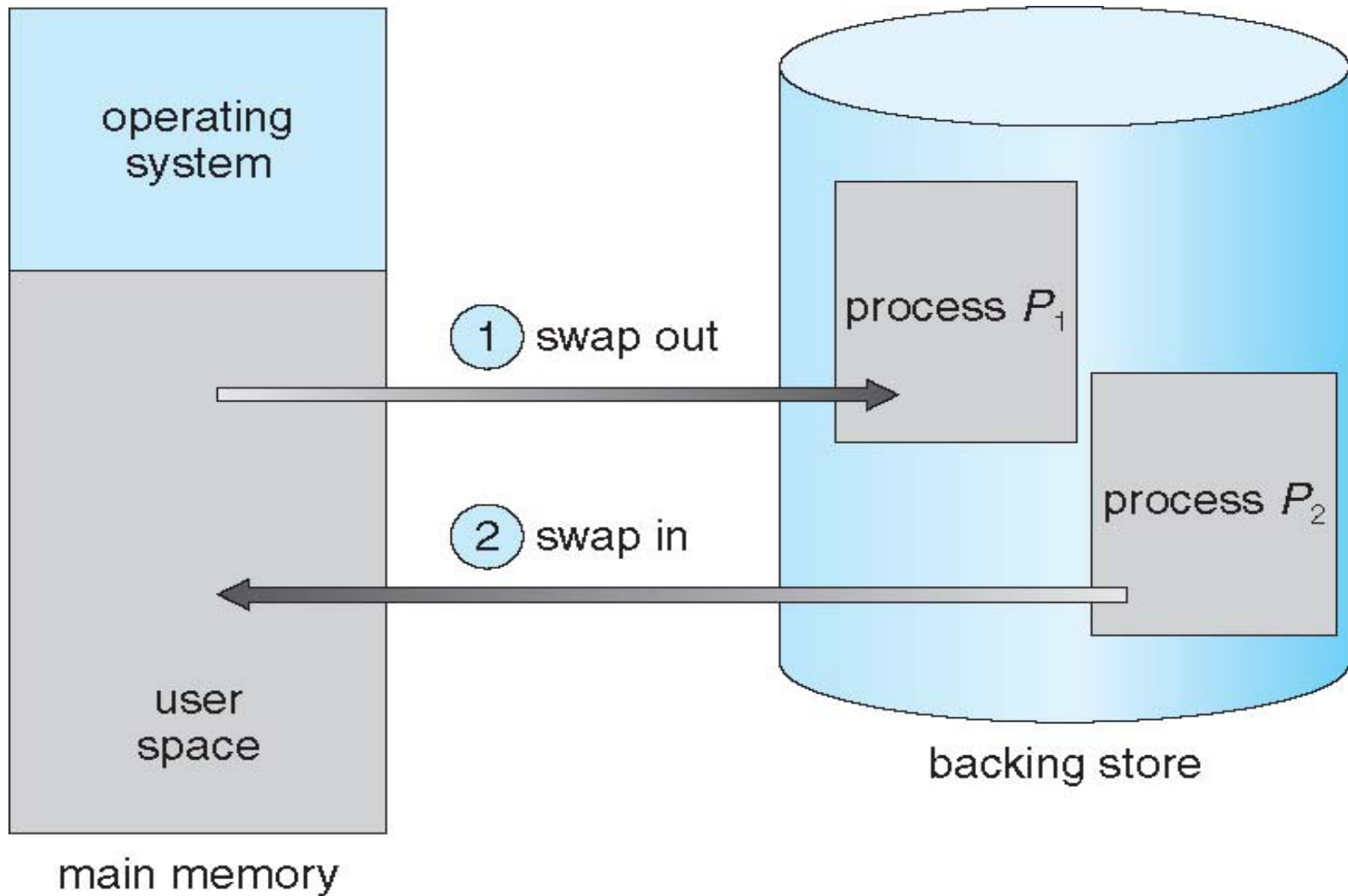


- **Stub:** đoạn mã nhỏ dùng để định vị các hàm thư viện ở trong bộ nhớ

Phần 7: Quản lý bộ nhớ

7.2 Swapping

Swapping



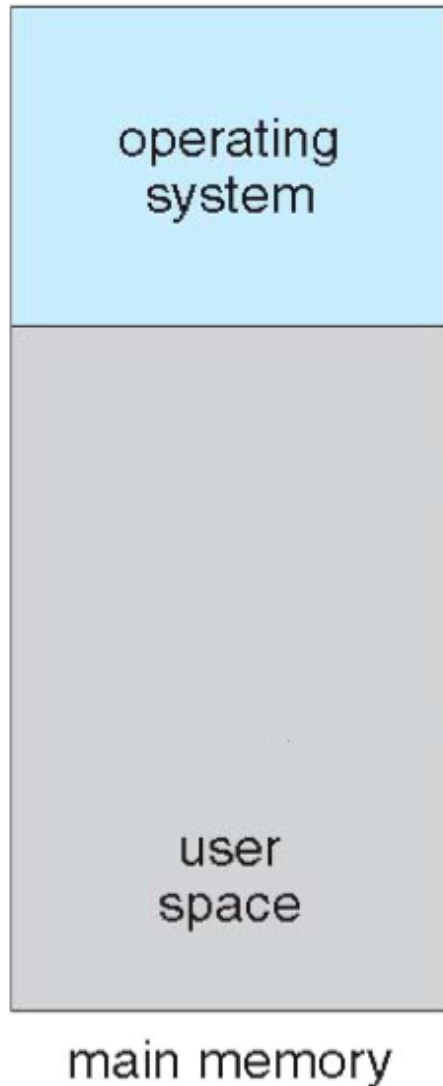
Swapping

- Cho phép dung lượng của tiến trình lớn hơn dung lượng bộ nhớ
- **Bộ lưu trữ phụ (backing store)**: đĩa có tốc độ truy cập nhanh và dung lượng đủ lớn để lưu tất cả hình ảnh bộ nhớ cho tất cả người dùng
- Thời gian chuyển trạng thái của hệ thống swapping tương đối lâu:
 - Ví dụ: tiến trình với dung lượng 100 MB cần chuyển vào bộ nhớ
 - Độ trễ của đĩa 8 ms $\rightarrow$ thời gian chuyển ra 2008 ms
 - Chuyển vào đĩa một tiến trình với cùng lượng $\rightarrow$ tổng thời gian 4016 ms
 - Nếu biết trước dung lượng bộ nhớ cần dùng, có thể giảm dung lượng bộ nhớ cần chuyển vào/ra $\rightarrow$ giảm thời gian chuyển trạng thái

Phần 7: Quản lý bộ nhớ

7.3 Cấp phát liên tục

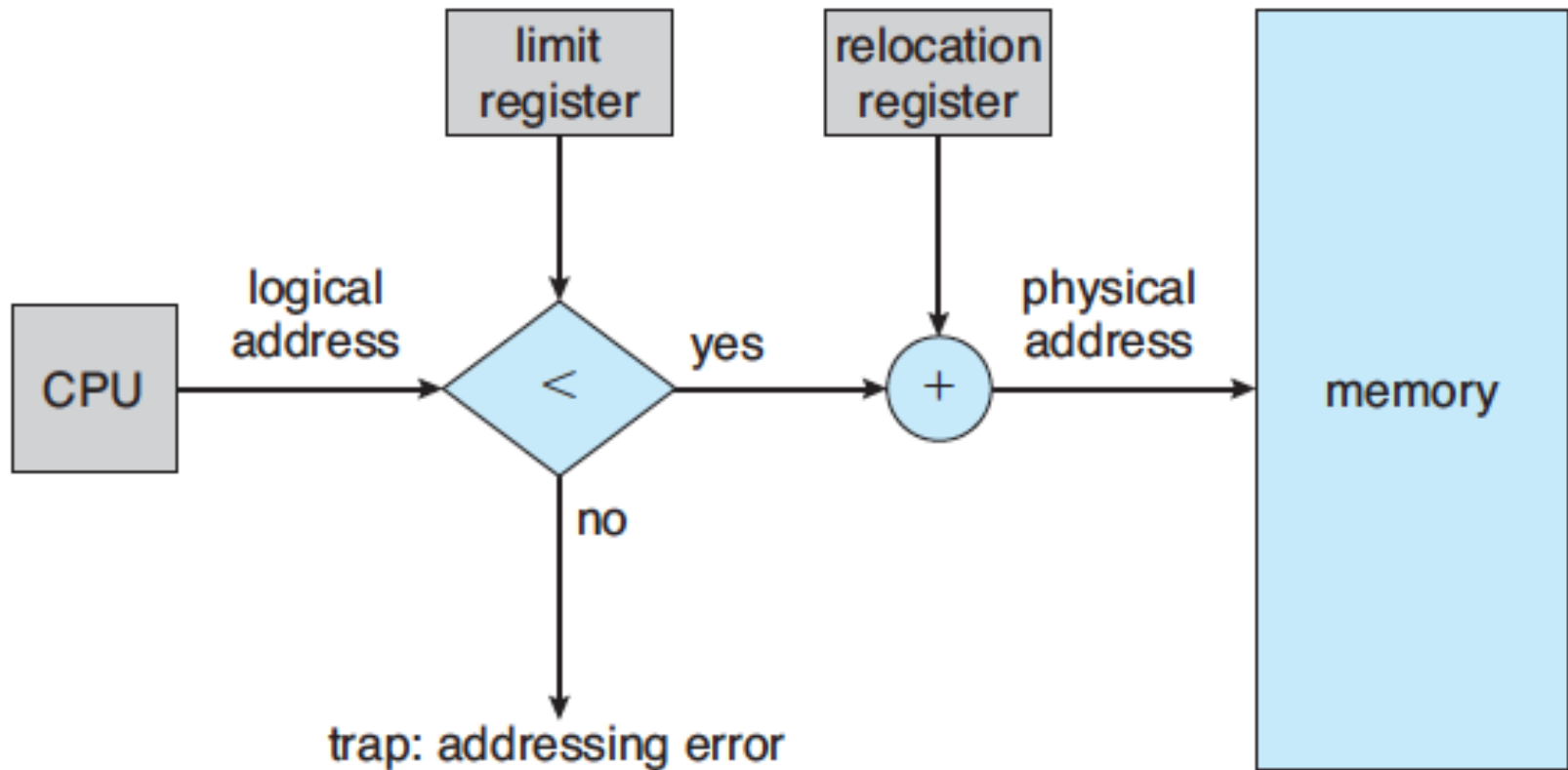
Bảo vệ bộ nhớ



- Mỗi tiến trình được lưu ở một khu vực liên tục trong bộ nhớ
- Các thanh ghi định vị lại giúp bảo vệ các tiến trình không truy cập khu vực bộ nhớ của nhau, thay đổi về mã và dữ liệu HĐH
 - Thanh ghi cơ sở: lưu địa chỉ vật lý nhỏ nhất
 - Thanh ghi giới hạn: giới hạn địa chỉ logic
 - MMU tự động ánh xạ các địa chỉ logic

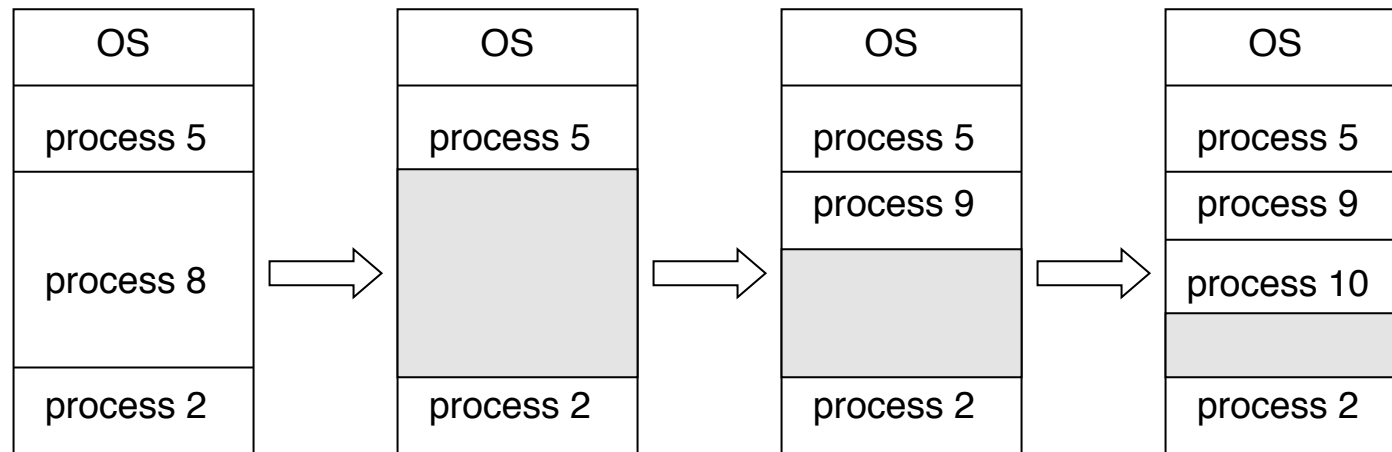
Phần cứng hỗ trợ thanh ghi định vị lại và thanh ghi giới hạn

19



Cấp phát bộ nhớ: Phương pháp đa phân vùng

- Bộ nhớ được chia làm các phân vùng với kích thước cố định
- Khi một phân vùng trống, tiến trình được lựa chọn từ input queue, rồi tải vào phân vùng đó
- Khi tiến trình kết thúc thì giải phóng phân vùng
- Cấp độ đa chương trình phụ thuộc vào số phân vùng của bộ nhớ



Cấp phát bộ nhớ: Bài toán cấp phát lưu trữ tự động

- HĐH dùng một bảng để lưu thông tin các phần bộ nhớ còn trống/ đang được sử dụng
- Các phần trống có kích thước khác nhau
- **Bài toán cấp phát lưu trữ tự động:** Làm thế nào để cấp phát phần bộ nhớ có kích thước n từ danh sách các khu vực trống
 - **First-fit:** Cấp phát khu vực trống đầu tiên đủ lớn
 - **Best-fit:** Cấp phát khu vực trống nhỏ nhất đủ lớn
 - **Worst-fit:** Cấp phát khu vực trống lớn nhất đủ lớn

First-fit và Best-fit nhanh hơn và có mức sử dụng bộ lưu trữ tốt Worst-fit

Hiện tượng phân mảnh

- **Phân mảnh ngoài**: tổng dung lượng bộ nhớ trống thoả mãn yêu cầu nhưng không liên tục
- **Phân mảnh trong**: trong phương thức đa phân vùng, dung lượng bộ nhớ cấp phát có thể lớn hơn dung lượng yêu cầu
- Khi sử dụng phương pháp First-fit, cứ cấp phát N khu vực thì $0.5 N$ khu vực khác bị mất do phân mảnh (**định luật 50%**)

Giải pháp khắc phục hiện tượng phân mảnh ngoài

■ Phương pháp nén

- Chuyển các ô nhớ sao cho tất cả phần bộ nhớ trống ở trong một khu vực liên tục
- Chỉ cho phép đổi với phương pháp định vị động xảy ra ở giai đoạn thực thi

■ Cho phép địa chỉ logic của các tiến trình không cần liên tục

- Phân đoạn
- Phân trang

Phần 7: Quản lý bộ nhớ

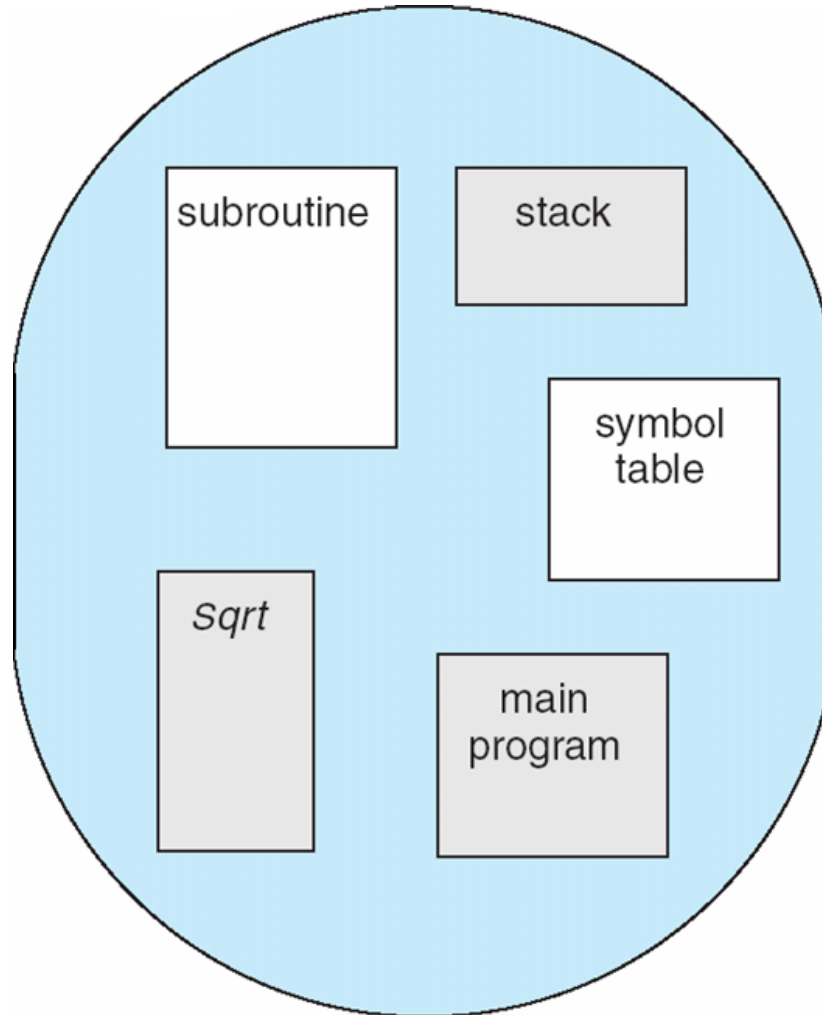
7.4 Phân đoạn

Phân đoạn là gì ?

- **Phân đoạn (segmentation)**: cách quản lý bộ nhớ theo cách nhìn của người dùng
- Mỗi chương trình là tập hợp các phân đoạn
- Mỗi phân đoạn là một đơn vị logic, ví dụ như chương trình chính, hàm, thủ tục, biến địa phương, biến toàn cục, mảng, ...

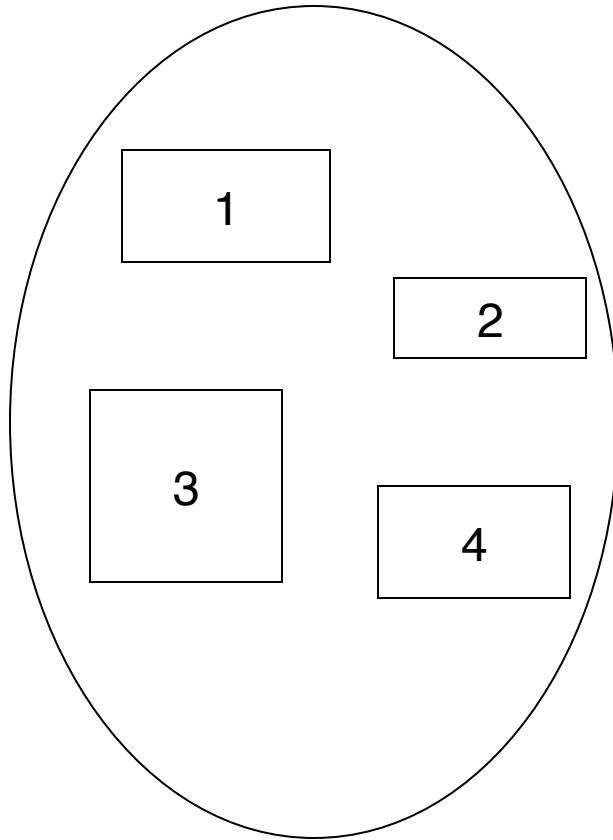
Cách nhìn của người dùng về một chương trình

26

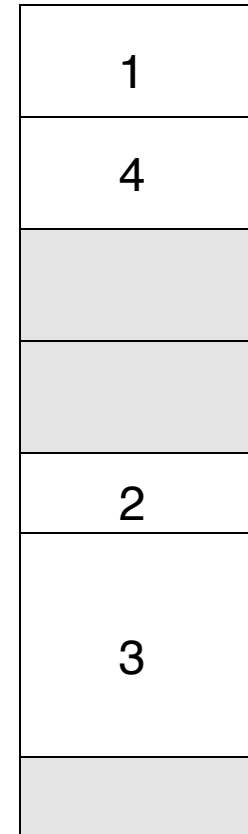


Cách nhìn logic của phân đoạn

27



user space



physical memory space

Kiến trúc phân đoạn

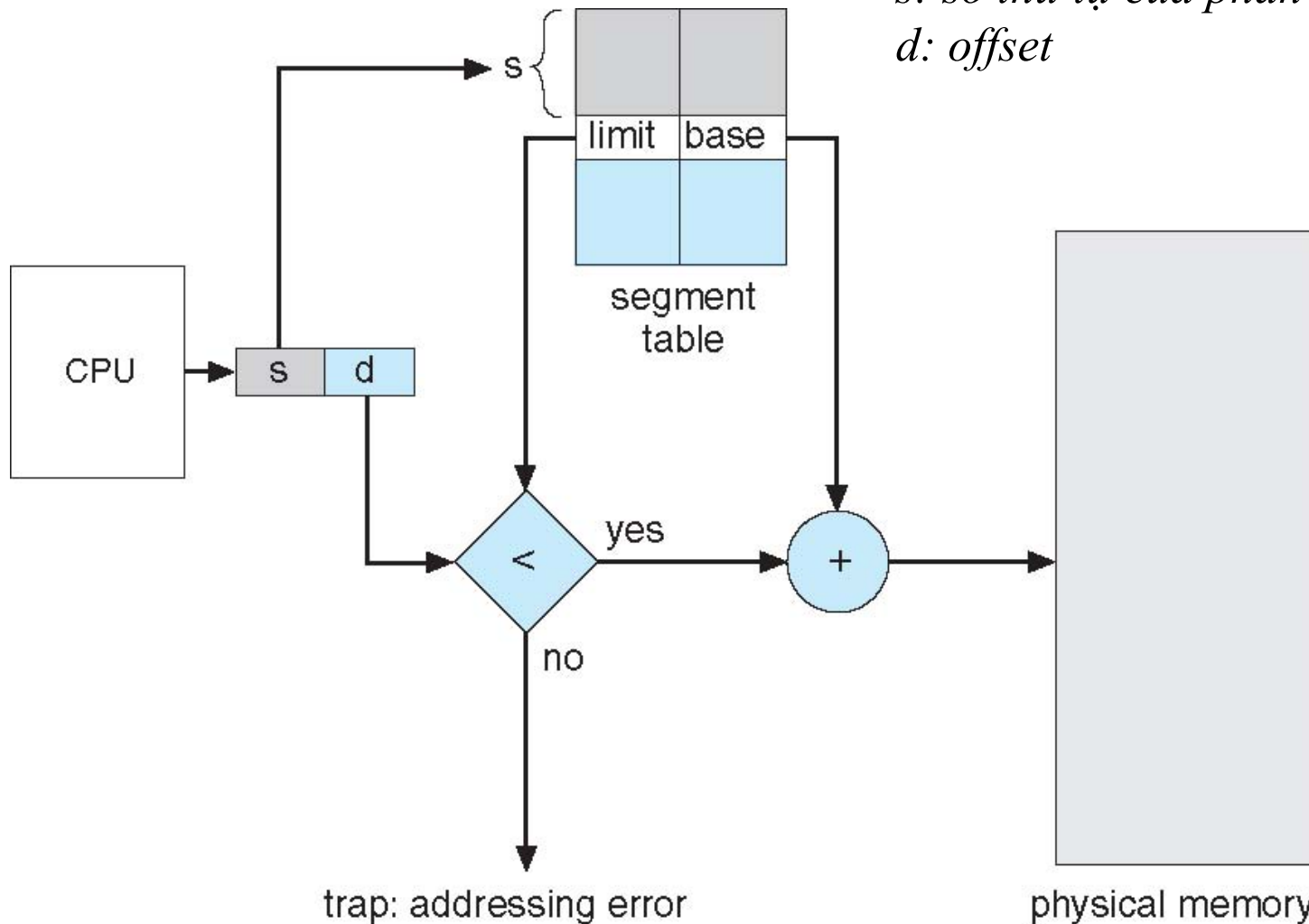
- Mỗi địa chỉ logic có những thông tin sau:
 $\langle \text{segment-number, offset} \rangle$
- **Bảng phân đoạn**: ánh xạ địa chỉ phân đoạn do người dùng định nghĩa (hai chiều) với địa chỉ vật lý (một chiều); mỗi hàng bao gồm:
 - **Cơ sở (base)**: địa chỉ vật lý bắt đầu của phân đoạn
 - **Giới hạn (limit)**: độ dài của phân đoạn
- **Thanh ghi cơ sở của bảng phân đoạn (STBR)**: chỉ đến vị trí bảng phân đoạn trong bộ nhớ
- **Thanh ghi độ dài của bảng phân đoạn (STLR)**: lưu số lượng phân đoạn sử dụng bởi một chương trình

Kiến trúc phân đoạn

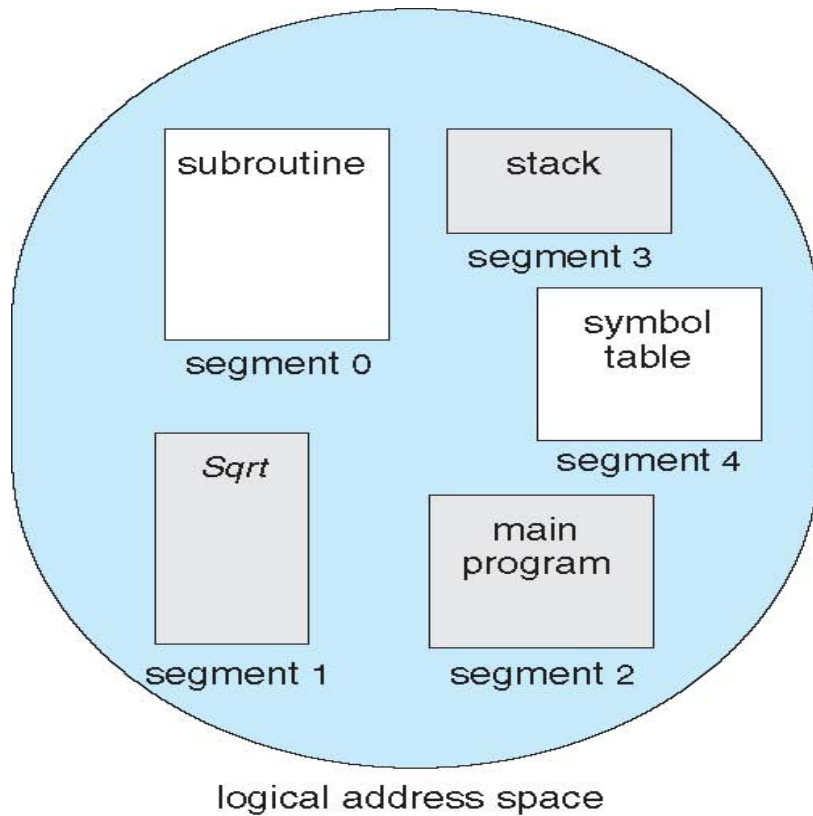
- **Bit bảo mật:** mỗi hàng trong bảng phân đoạn có những bit bảo mật sau
 - Bit xác nhận (bằng 0 nếu số thứ tự của phân đoạn có ở trong bảng phân đoạn)
 - Quyền đọc / viết / thực thi
- **Nhược điểm:** Không tránh được hiện tượng phân mảnh ngoài

Phần cứng hỗ trợ phân đoạn

s: số thứ tự của phân đoạn
d: offset

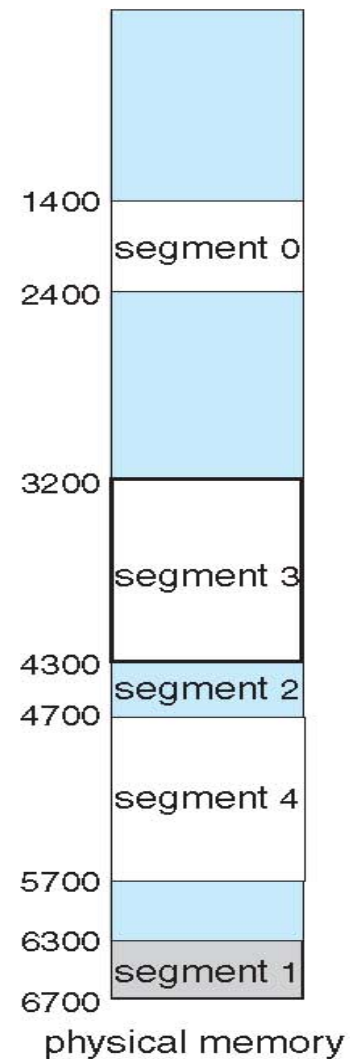


Ví dụ phân đoạn



	limit	base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

segment table



Phần 7: Quản lý bộ nhớ

7.5 Phân trang

Phân trang là gì ?

- Bộ nhớ vật lý được chia thành các khối có cùng kích cỡ, 512 bytes đến 16 MB ([frame](#))
- Bộ nhớ logic được chia thành các khối có cùng kích cỡ ([page](#))
- Sử dụng bảng phân trang để ánh xạ địa chỉ logic với địa chỉ vật lý
- Bộ lưu trữ phụ cũng được chia thành các trang

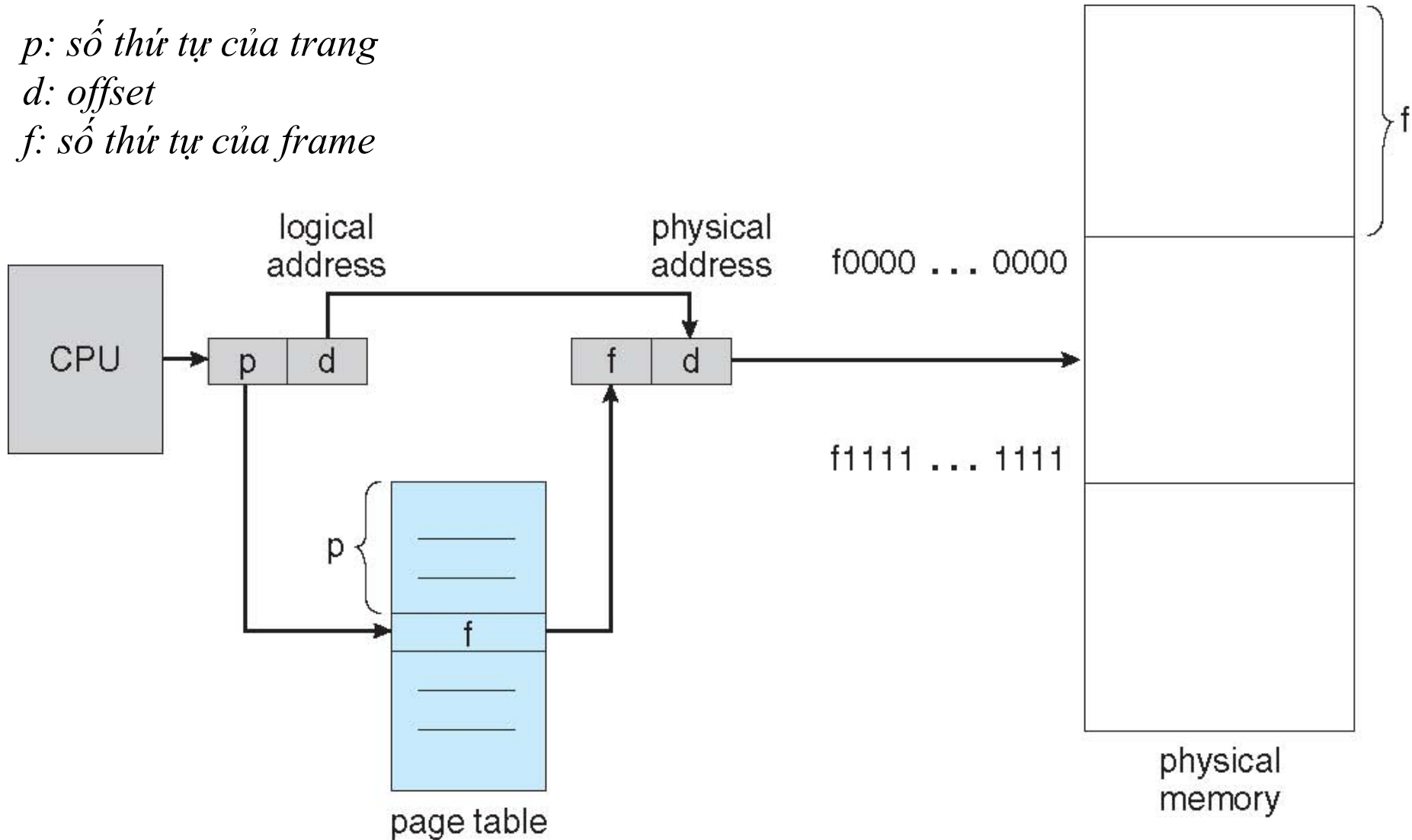
Phần cứng phân trang



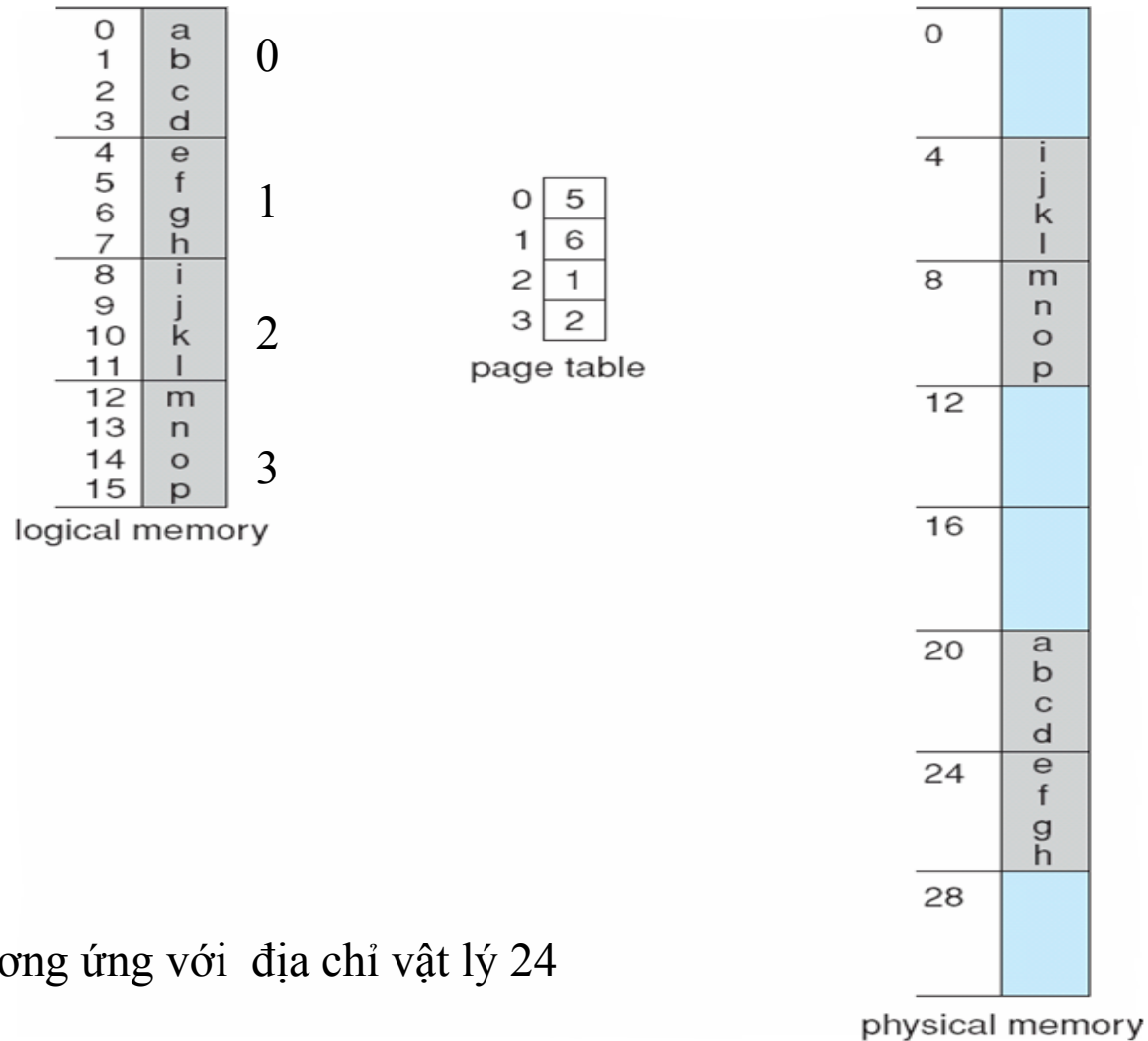
p: số thứ tự của trang

d: offset

f: số thứ tự của frame



Ví dụ phân trang



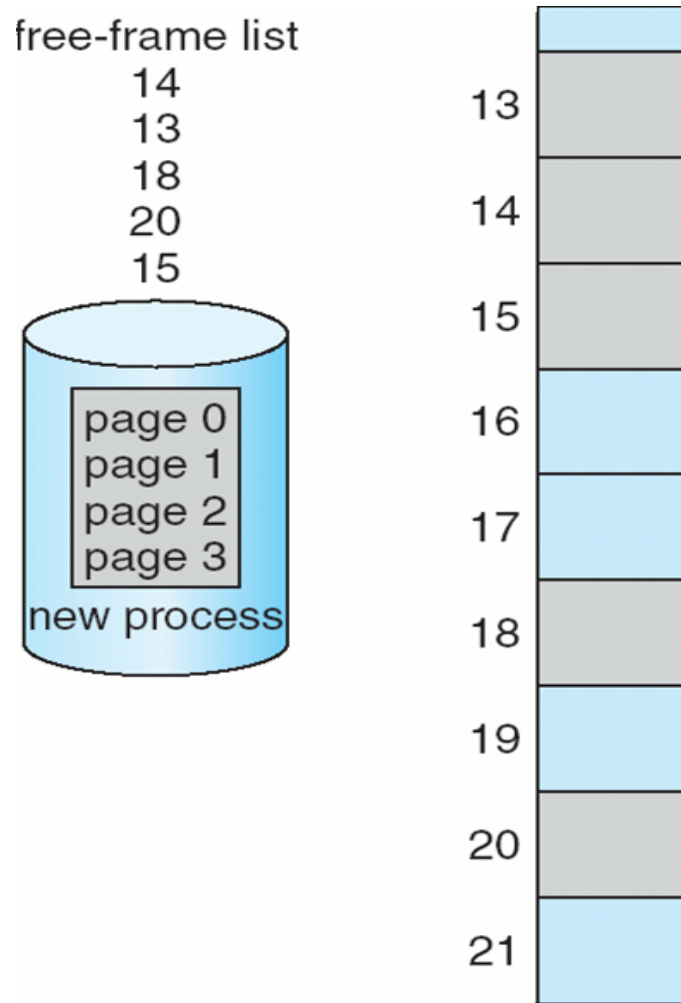
Địa chỉ logic 4 tương ứng với địa chỉ vật lý 24

trang 4-byte và bộ nhớ 32-byte (8 trang)

Hiện tượng phân mảnh

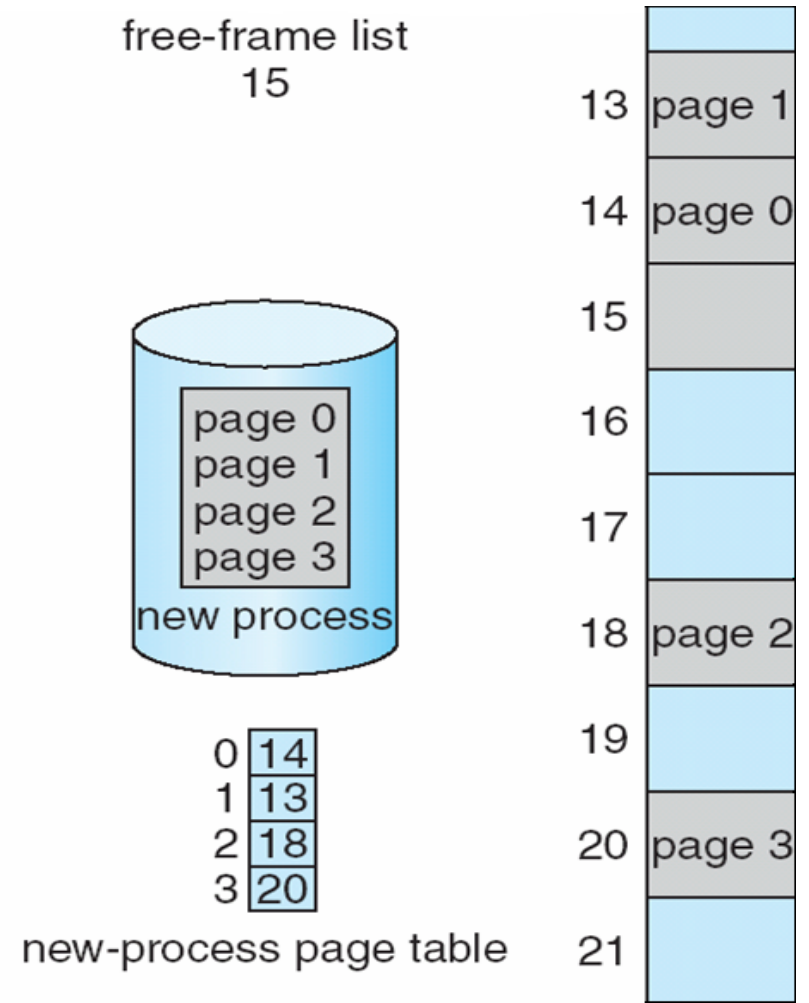
- Phương thức phân trang tránh được hiện tượng phân mảnh ngoài
 - Bất cứ frame nào còn trống sẽ được phân cho tiến trình khi có yêu cầu
- Nhưng có thể xảy ra phân mảnh trong:
 - Ví dụ: kích thước mỗi frame là n byte, kích thước của tiến trình là $(k.n + m)$ byte, trong đó $m < n$. Nghĩa là phải cung cấp $(k+1)$ frame cho tiến trình $\rightarrow$ xảy ra hiện tượng phân mảnh trong $(n-m)$ byte

Quản lý frame trống



(a)

trước khi cấp phát



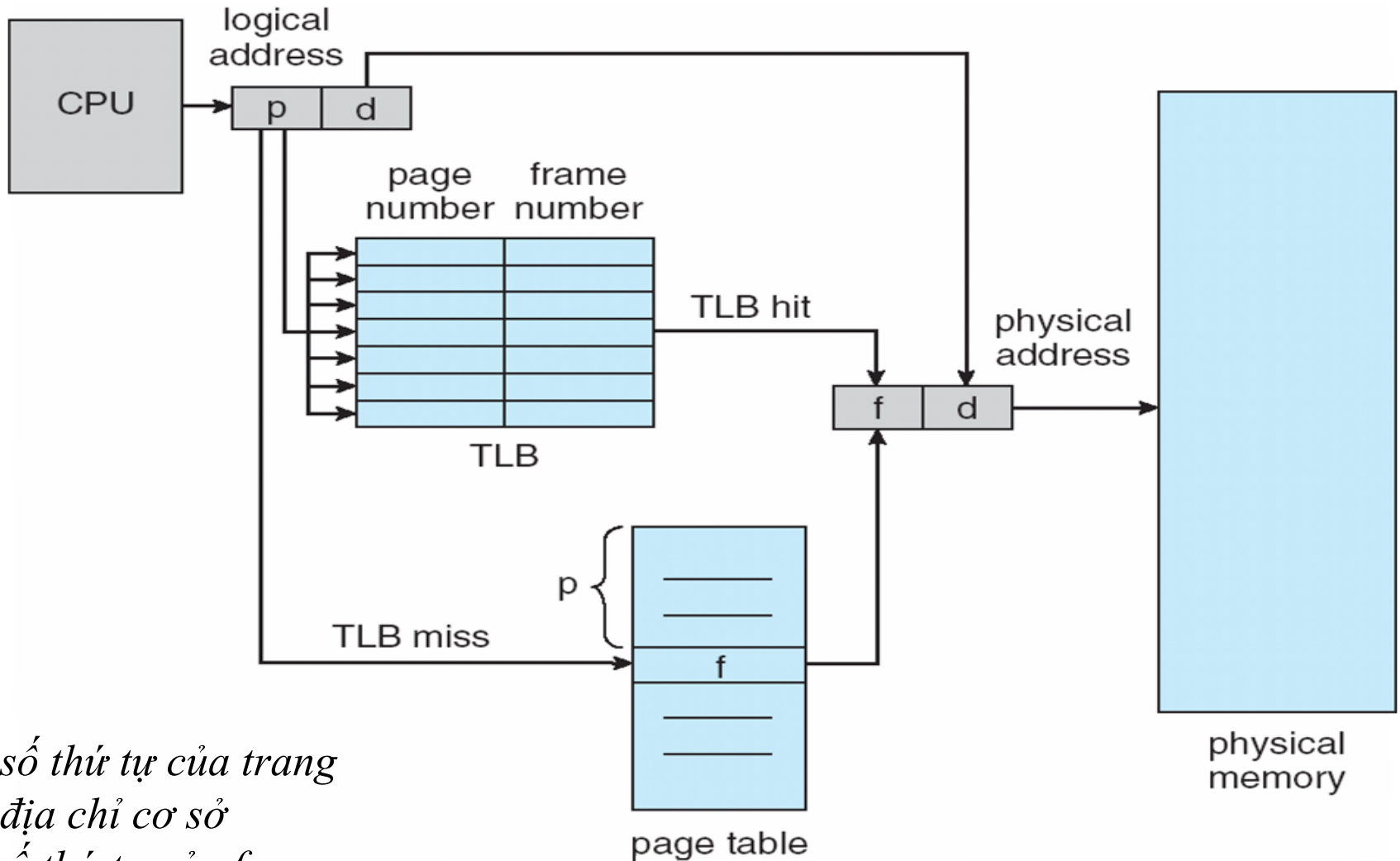
(b)

sau khi cấp phát

Bảng phân trang

- Đối với bảng phân trang lớn, bảng phân trang được lưu trong bộ nhớ
- Thanh ghi cơ sở bảng phân trang: chỉ đến vị trí bảng phân trang trong bộ nhớ
- Thanh ghi độ dài bảng phân trang: lưu độ dài của bảng
- Mỗi truy vấn câu lệnh / dữ liệu cần hai truy vấn bộ nhớ: một cho bảng phân trang, một cho dữ liệu / câu lệnh
- Sử dụng cache hỗ trợ tìm kiếm đặc biệt *translation look-aside buffer (TLB)* có kích thước 64 -1024 hàng

Phần cứng hỗ trợ phân trang với TLB



p: số thứ tự của trang
d: địa chỉ cơ sở
f: số thứ tự của frame

Thời gian truy cập bộ nhớ hiệu quả

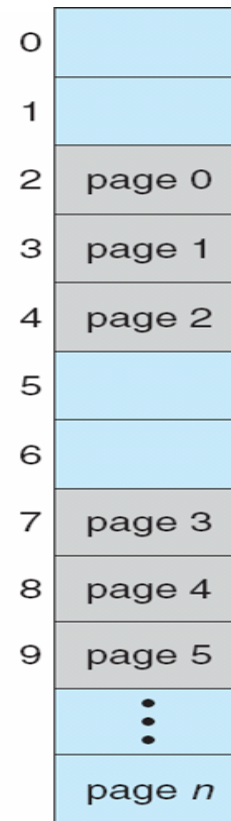
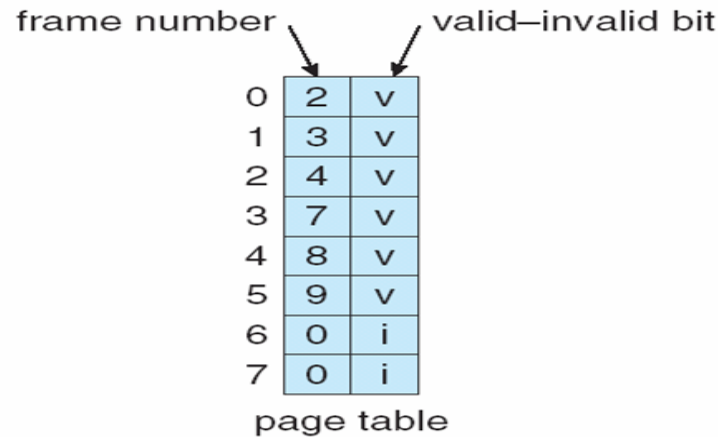
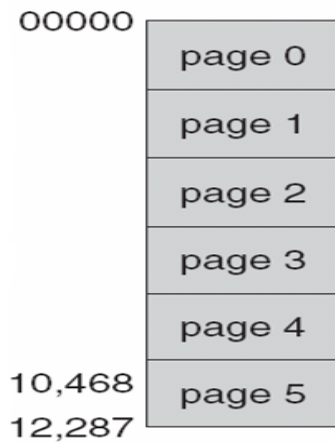
- Tỷ lệ trúng (Hit ratio) = % số lần tìm được trang cần thiết ở TLB
- Ví dụ: Tỷ lệ trúng $\alpha=80\%$, thời gian truy cập bộ nhớ 100 ns. Nếu trang ở trong TLB, thì cần thời gian truy cập bộ nhớ liên kết là 100 ns. Nếu trang không có trong TLB, thì cần truy cập bộ nhớ để lấy số thứ tự của trang và frame, rồi truy cập phần bộ nhớ cần thiết (tổng cộng 200 ns)

$$\begin{aligned}\text{Thời gian truy cập bộ nhớ hiệu quả} &= 0.8 \times 100 + 0.2 \times 200 \\ &= 120 \text{ ns}\end{aligned}$$

Bit bảo mật

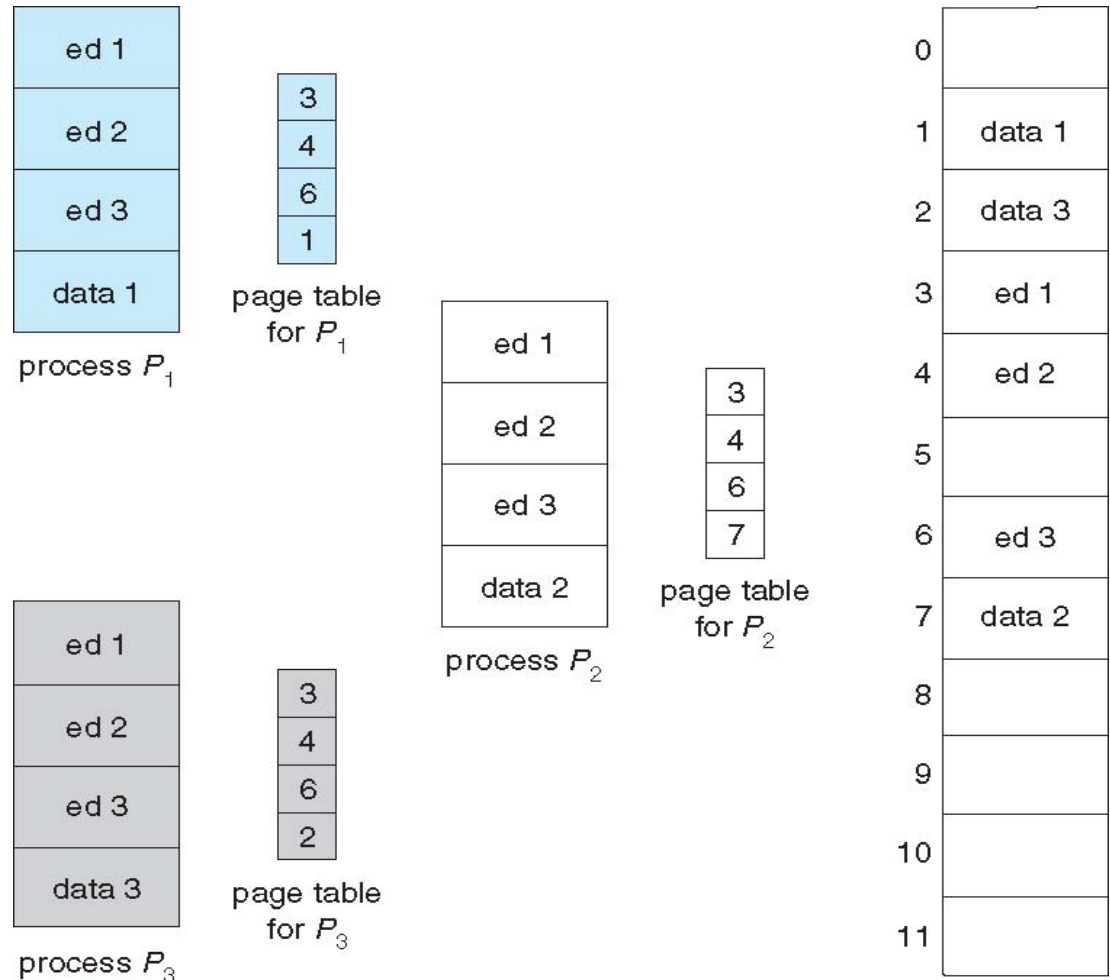
Mỗi trang có bit bảo mật sau:

- Quyền đọc / viết / thực thi
- Bit xác nhận (bằng 1 nếu trang có trong không gian địa chỉ logic của tiến trình)



Trang chia sẻ

- Một số tiến trình chia sẻ một bản sao của mã lệnh

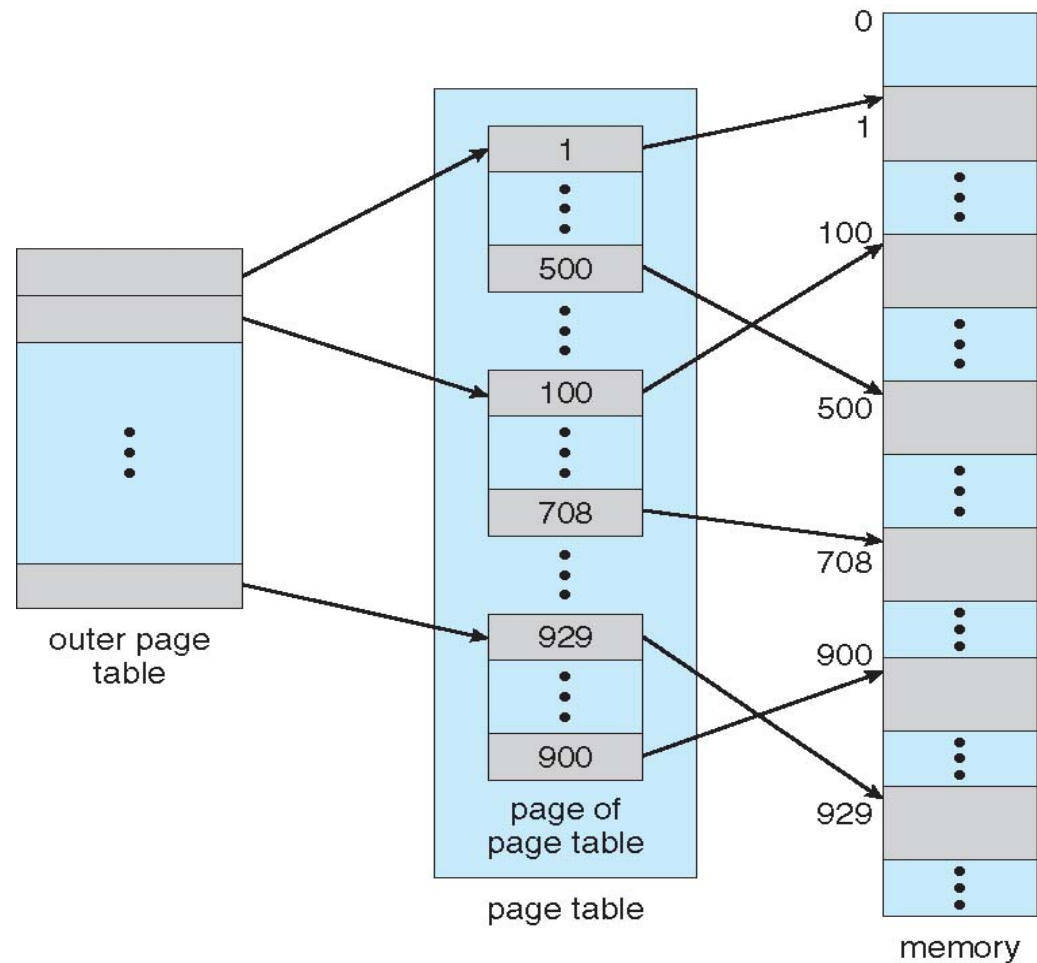


Phần 7: Quản lý bộ nhớ

7.6 Cấu trúc bảng phân trang

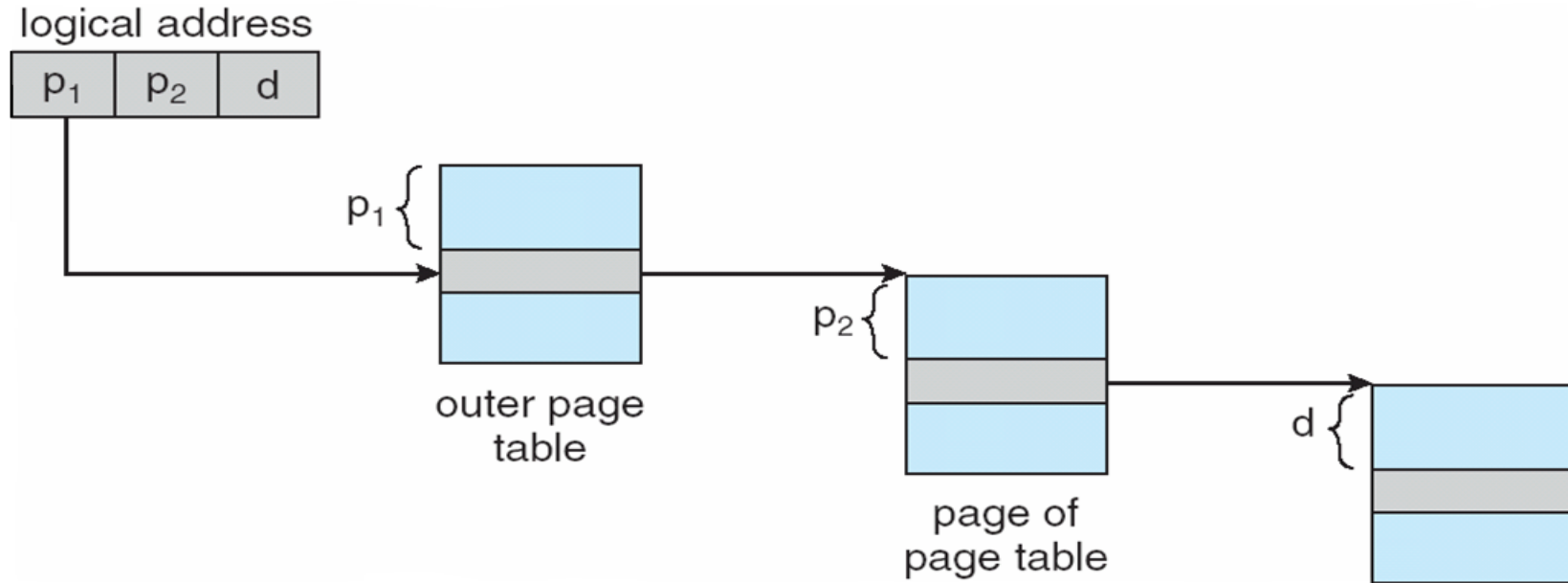
Cấu trúc phân tầng

- Không gian địa chỉ logic có thể được phân làm nhiều bảng trang

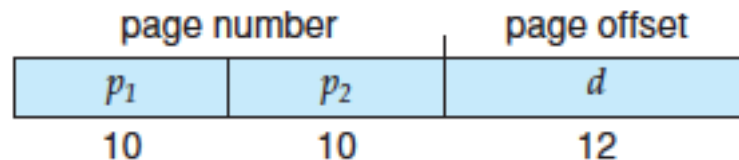


Cấu trúc phân tầng: biên dịch địa chỉ

45

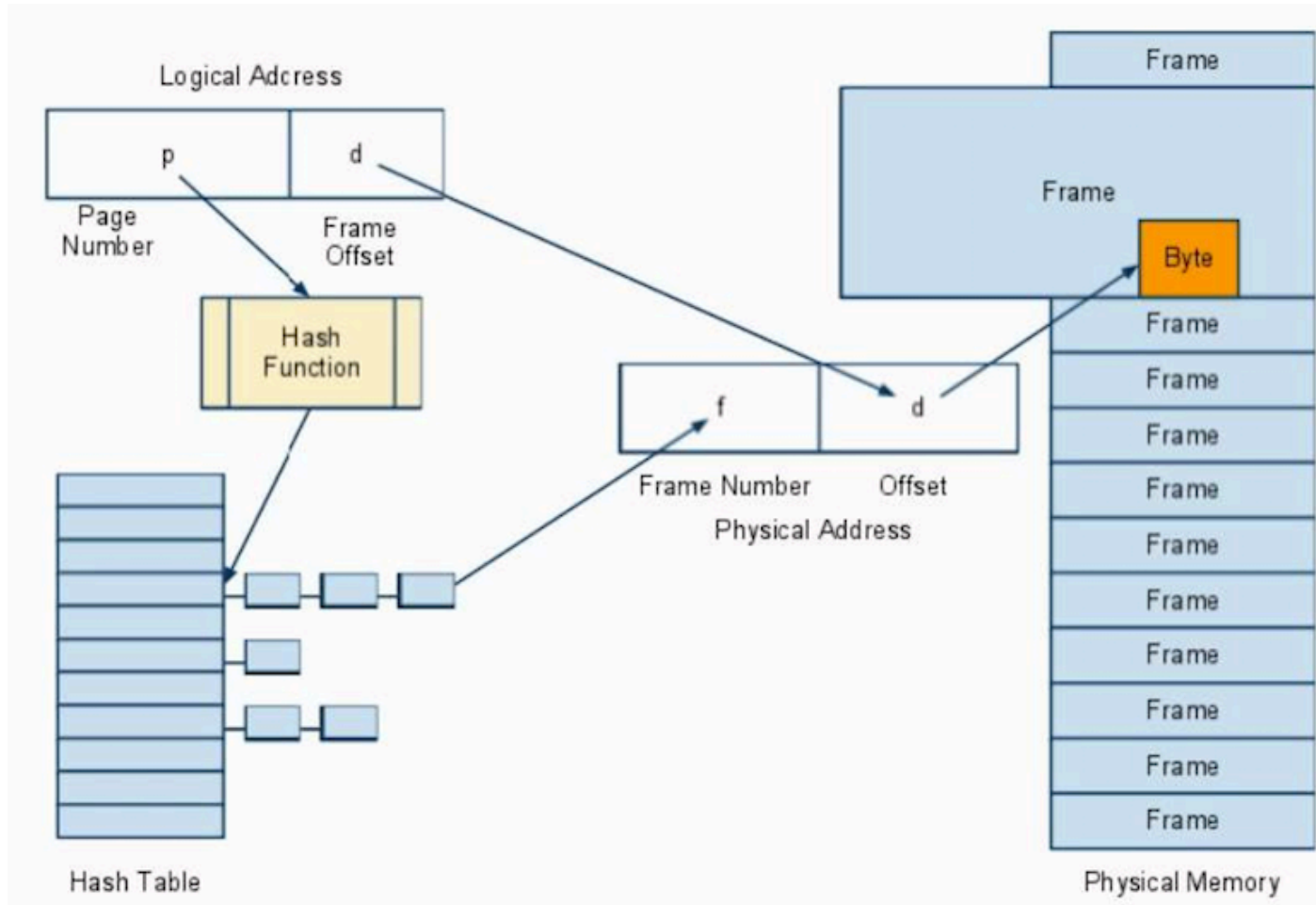


- Ví dụ địa chỉ logic 32 bit, kích thước trang 4 KB có thể được chia thành



Cấu trúc băm

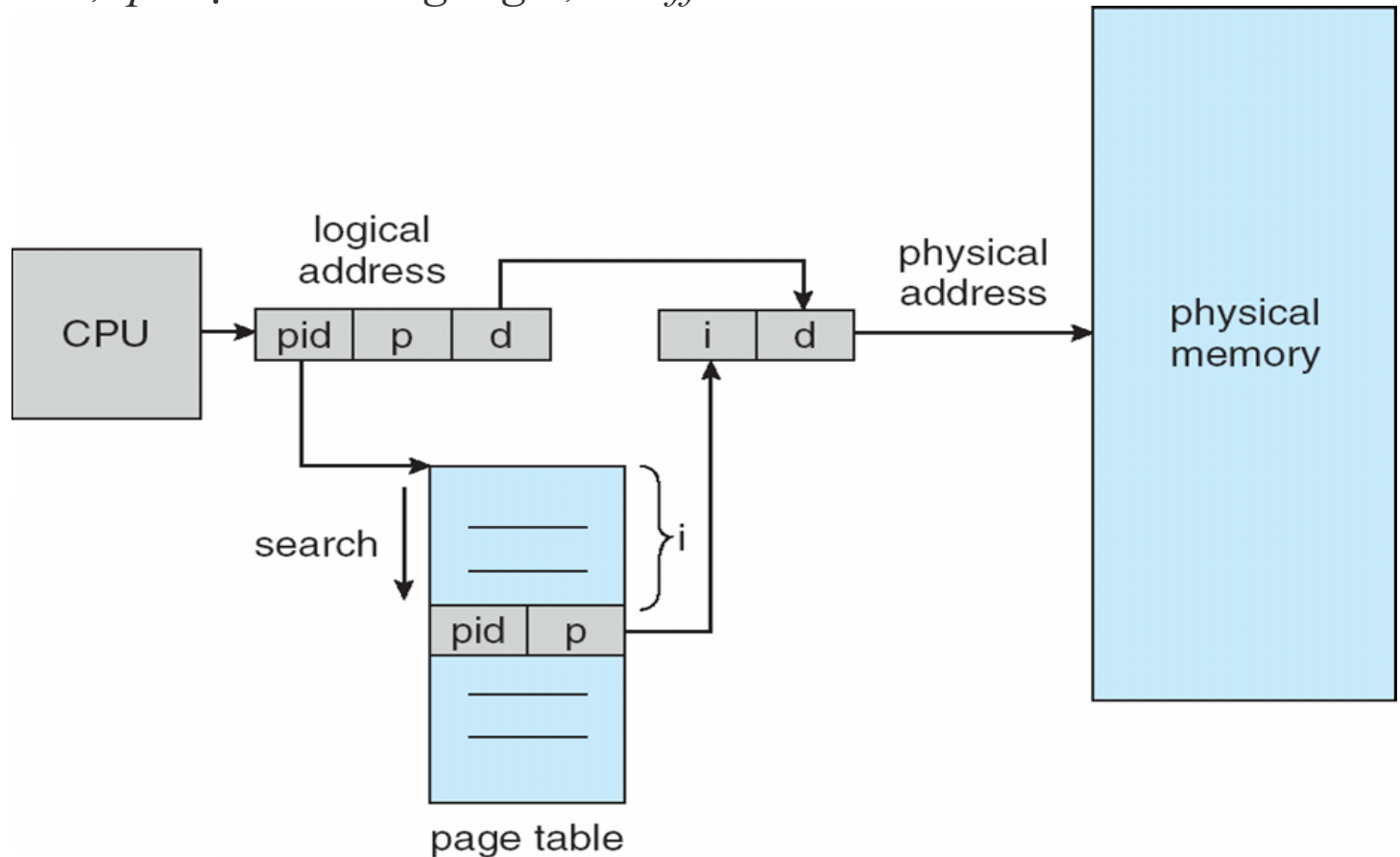
- Thường sử dụng cho hệ máy có không gian địa chỉ > 32 bit



Cấu trúc nghịch đảo

Hàng thứ N chứa thông tin của frame thứ N trong bộ nhớ vật lý

pid: số hiệu tiến trình, *p*: địa chỉ trang logic, *d*: offset



Bài tập 1

Hệ thống sử dụng 12 bit cho địa chỉ ảo và địa chỉ vật lý, kích thước mỗi trang là 256 byte. Dựa vào bảng phân trang sau:

Page no.	Frame no.	
0	–	a. Tìm địa chỉ vật lý tương ứng cho các địa chỉ ảo sau (dạng hexa): 9EF, 700, 0FF
1	2	
2	12	
5	–	b. Nếu kích thước mỗi đơn vị bộ nhớ là 4 byte, hãy tính dung lượng bộ nhớ chính, số lượng frame
3	10	
4	–	
5	4	
6	3	
7	–	
8	11	
9	0	

Bài tập 2

Ánh xạ bộ nhớ ảo 1 GB lên bộ nhớ vật lý có 256 frame, mỗi frame có kích thước 4 KB. Kích thước mỗi đơn vị bộ nhớ là 1 byte.

- Liệu bộ nhớ trong chứa được toàn bộ bảng phân trang ?
Giải thích.
- Liệu bảng phân trang nghịch đảo chứa được trong một trang ? Giải thích